Dear Candidate,

As part of our technical evaluation process, we would like you to complete the following Laravel assessment.

The purpose of this assessment is not only to evaluate project completion, but also to understand your approach to Laravel development, database design, business logic, code structure, security, performance, and maintainability.

Project Description: Support Ticket & SLA Management System

Develop a Support Ticket Management System using Laravel.

The application should allow users/customers to create support tickets and support agents to manage, respond to, assign, resolve, and track those tickets.

1. Ticket Management

Implement ticket management functionality including:

- Create ticket
- View ticket
- Update ticket
- Resolve/close ticket
- View upcoming/active tickets
- View resolved/closed tickets

Each ticket should contain appropriate information such as:

- Unique Ticket ID
- Subject
- Description
- Priority
- Category
- Status
- Assigned Agent
- Created By
- Created Date
- SLA/Due Time

Example statuses may include:

- Open

- In Progress
- Waiting for Customer
- Resolved
- Closed

You may extend or modify the statuses if your design requires it.

2. Ticket ID Generation

Generate a unique ticket number using a predefined prefix format.

Example:

TKT-2026-000001

The implementation should properly handle uniqueness and future growth.

3. Roles and Permissions

The system should contain at least the following roles:

- Administrator
- Support Agent
- Customer/User

Expected authorization behaviour includes:

- Customers should only be able to access their permitted tickets.
- Support agents should only perform actions they are authorized to perform.
- Administrators should have appropriate administrative access.
- Authorization must be enforced on the server side.

You are free to design the permission structure.

4. Ticket Assignment

Tickets should be assignable to support agents.

Implement:

- Manual ticket assignment
- Automatic ticket assignment

You may determine the strategy for automatic assignment.

For example, you may consider:

- Agent workload
- Round-robin assignment
- Ticket category
- Agent availability
- Any other reasonable strategy

Please document your chosen approach.

5. Concurrent Ticket Assignment

Consider the following scenario:

Two support agents attempt to accept the same unassigned ticket at approximately the same time.

The system must ensure that the ticket is not incorrectly assigned to multiple agents.

You should design an appropriate solution and be prepared to explain your implementation.

6. Ticket Conversation

Users and support agents should be able to communicate through the ticket.

The system should support:

- Customer replies
- Agent replies
- Internal agent notes

Internal notes must never be visible to customers.

7. Attachments

Users and agents should be able to attach files to tickets or replies.

Appropriate consideration should be given to:

- File validation
- File storage
- File size/type restrictions
- Secure file access

8. SLA Management

Tickets should have a resolution SLA based on their priority.

Example:

Priority	Resolution Time
Low	48 Hours
Medium	24 Hours
High	8 Hours
Critical	2 Hours

The application should identify tickets such as:

- Within SLA
- Approaching SLA breach
- SLA breached

The SLA configuration should be designed so that changing SLA rules does not require unnecessary changes throughout the application.

9. Business Hours

SLA calculation should consider configurable business hours.

Example:

Business Days: Sunday–Thursday
Business Hours: 9:00 AM–6:00 PM

For example:

If a Critical ticket with a 2-hour SLA is created on Thursday at 5:00 PM, simply adding two clock hours would not correctly calculate its SLA deadline.

You should design an appropriate approach for calculating the deadline.

The solution does not need to cover every possible enterprise calendar scenario, but your implementation and reasoning should be clear and maintainable.

10. SLA Escalation

The application should automatically identify and escalate tickets that are approaching or have exceeded their SLA.

An escalation may include:

- Marking the ticket as escalated
- Notifying appropriate users
- Sending an email
- Creating an escalation activity/history entry

You may determine the exact escalation mechanism.

11. Notifications

Send notifications for important ticket events.

Examples:

- Ticket created
- Ticket assigned
- Agent replied
- Customer replied
- Ticket resolved
- SLA approaching
- SLA breached

Email notifications should be processed appropriately without unnecessarily blocking user requests.

12. Search and Filtering

Provide ticket search and filtering functionality.

Examples:

- Ticket ID
- Keyword
- Status
- Priority
- Category
- Assigned agent
- Customer
- Created date
- SLA status

Pagination should be implemented.

13. CSV Import

Provide a way to import ticket data from CSV.

The import should be designed to reasonably support large files.

Consider:

- Memory usage
- Validation
- Invalid rows
- Duplicate data
- Error reporting
- Long-running processing

You may decide the appropriate Laravel implementation.

14. Dashboard

Create a simple but useful dashboard.

It may contain information such as:

- Total open tickets
- Unassigned tickets
- Critical tickets
- SLA breached tickets
- Tickets approaching SLA breach
- Resolved tickets
- Tickets by priority
- Tickets by agent

The dashboard does not need to contain complex visualizations.

We are more interested in usability, query quality, and implementation.

15. Activity / Audit History

Important ticket activities should be traceable.

Example:

10:01 AM – Ticket created by User A
10:05 AM – Assigned to Agent B
10:20 AM – Priority changed from Medium to High
10:35 AM – Agent replied
11:00 AM – Ticket escalated
11:15 AM – Status changed from In Progress to Resolved

The activity history should provide enough information to understand what happened to a ticket.

16. Database and Performance Considerations

Design the database assuming that the application may eventually contain a large number of:

- Tickets
- Ticket replies
- Attachments
- Activity records
- Notifications

We will evaluate areas such as:

- Database structure
- Relationships
- Indexes
- Query efficiency
- Pagination
- N+1 query handling
- Eloquent usage

You are not required to generate millions of records.

17. Automated Testing

Include automated tests for areas you consider important.

We intentionally do not specify exactly which tests must be written.

Part of the evaluation is understanding which areas you consider critical enough to test.

Technical Expectations

You are free to determine the architecture of the application.

We do not require you to use specific design patterns or Laravel components simply for the sake of using them.

Use Laravel features where you believe they are appropriate.

Your technical decisions should be reasonable, maintainable, and explainable.

We may evaluate areas such as:

- Validation

- Eloquent relationships
- Authorization
- Business logic organization
- Jobs and queues
- Events/listeners
- Notifications
- Scheduler
- Database transactions
- Error handling
- Storage
- Configuration
- Testing
- Security
- Performance

AI Tool Usage

You are allowed to use AI-assisted development tools such as:

- ChatGPT
- Claude
- GitHub Copilot
- Cursor
- Other similar tools

Using AI tools will not negatively affect your assessment.

However, you are fully responsible for every piece of code included in your submission.

During the technical interview, you may be asked to:

- Explain any part of your implementation.
- Explain why a particular architectural or database decision was made.
- Debug an issue in your submitted project.
- Modify an existing feature.
- Add a small new business requirement.
- Refactor part of the submitted code.
- Update or write tests for a change.

Therefore, submitting AI-generated code that you do not understand may significantly affect your evaluation.

Evaluation Criteria

Project Completion

The candidate will not automatically receive a negative evaluation if the project is not 100% completed.

We value the quality of completed functionality more than superficially implementing every requirement.

Laravel Knowledge

We will evaluate your understanding and practical use of Laravel.

Code Structure

We will assess:

- Application organization
- Separation of concerns
- Maintainability
- Reusability
- Appropriate abstraction

Business Logic

We will evaluate how effectively you translate requirements such as:

- SLA calculation
- Escalation
- Ticket assignment
- Authorization
- Business hours

into maintainable application logic.

Database Management

We will evaluate:

- Schema design
- Relationships
- Indexing
- Query efficiency
- Data integrity

- Appropriate use of transactions where necessary

Security and Authorization

We will evaluate whether users can access only the resources and actions they are authorized to access.

Background Processing

We will evaluate how appropriately long-running or asynchronous tasks are handled.

Testing

We will evaluate both the quality of the tests and your choice of what should be tested.

Code Quality

We will assess:

- Readability
- Naming
- Consistency
- Maintainability
- Error handling
- Avoidance of unnecessary complexity

UI/UX

The application should be reasonably clean, responsive, and usable.

UI design is not the primary focus of this assessment.

Bonus

You may include additional improvements that demonstrate your skills.

Examples may include:

- Docker setup
- API endpoints
- API documentation
- CI/CD
- Caching
- Real-time updates
- Import progress tracking

- Advanced reporting
- Rate limiting
- Additional automated tests
- Logging/monitoring
- Other meaningful engineering improvements

Bonus features should not come at the cost of poor implementation of the core requirements.

Submission Requirements

Please provide:

1. A GitHub repository containing the complete source code.
2. A proper README or documentation containing complete instructions for running the project locally.
3. Database setup/migration instructions.
4. Required environment/configuration instructions.
5. Queue/scheduler setup instructions if they are required by your implementation.
6. Test execution instructions.
7. Any important architectural decisions or assumptions you made.

A submission may be rejected if sufficient instructions are not provided to set up and run the project.

Please do not include production credentials, API keys, passwords, or other secrets in the repository.

You may optionally deploy the application and provide a production/demo URL.

Deployment is considered a plus but is not mandatory.

Technical Interview

After reviewing the submission, we will conduct a technical interview.

The interview will cover a range of technical topics, including discussions about your submitted project. You should be prepared to explain your implementation, answer technical questions, and make a small change to the application during the interview. The change may involve modifying a business rule, fixing an issue, changing authorization behavior, improving a query, or adding/updating a test. The purpose of this exercise is to evaluate your understanding of the code and your ability to work with an existing codebase rather than your ability to memorize Laravel syntax.